Prof. Gustavo Lopes Tamiosso
Prof. Vinicius José Fritzen

Trabalho Final — Expedição na Ilha dos Algoritmos

Desenvolvimento em dupla

Visão geral. Desenvolva, em C, um jogo de exploração por turnos. O jogador controla um personagem que atravessa um mapa, coleta cristais e tenta alcançar a saída antes de perder toda a energia. A dupla escolhe **uma** das duas interfaces abaixo e a usa por completo:

- **ncurses:** interface textual, desenhada no terminal com caracteres (Anexo A).
- **Raylib:** interface gráfica em janela, desenhada com formas simples (Anexo B).

As duas opções são igualmente aceitas, valem os mesmos pontos e são avaliadas pelos mesmos critérios. A escolha é feita uma única vez e registrada no README. Ela não altera os requisitos de mapa, estado, movimento, eventos, vitória, derrota, repetição e testes. A Raylib exige um pouco mais de trabalho (laço de quadros e desenho gráfico) e **não vale nota extra**. A proposta exercita matrizes, strings, funções, ponteiros, registros (`struct`), arquivos, modularização em múltiplos arquivos, testes e integração de um programa maior. O projeto deve ser menor e mais simples que um jogo completo.

Peso. Conforme o plano de ensino, o Trabalho Final (TF) vale 10% da Média Final ($MF = 0{,}3 \times P1 + 0{,}45 \times P2 + 0{,}15 \times AP + 0{,}10 \times TF$) e é feito como atividade autônoma, fora do horário de aula. Ele foi dimensionado para isso: priorize a versão-base bem feita.

Como ler. As Seções 1 a 14 valem para todas as duplas. Dos anexos, leia apenas o da interface escolhida.

Sumário. 1) Regras do jogo; 2) Requisitos obrigatórios; 3) Ambiente de desenvolvimento e correção; 4) Interfaces; 5) Arquivo de mapa; 6) Tela de fim de jogo; 7) Laço principal; 8) Módulos e integração; 9) Extensões opcionais; 10) Entregas; 11) Prazos; 12) Testes mínimos; 13) Avaliação; 14) Uso de IA e integridade acadêmica; Anexo A) ncurses; Anexo B) Raylib.

1. Objetivo e regras do jogo

O jogador começa na posição marcada com `P` no arquivo de mapa, com a energia inicial definida no arquivo e zero cristais. Em cada turno, escolhe uma direção usando as setas (ou teclas definidas pela dupla e documentadas no README).

1. O jogador não pode sair dos limites do mapa nem atravessar obstáculos (`#`).
2. Cada movimento válido consome uma unidade de energia e conta um turno.
3. Movimento inválido não altera a posição, não consome energia e **não** conta turno; o jogo mostra uma mensagem.
4. Ao entrar em uma casa com cristal, o jogador coleta o cristal, e a casa vira `.`.
5. Ao entrar em uma casa de evento, o jogo aplica o efeito, informa o jogador e a casa vira `.` (cada evento é ativado uma única vez).
6. A energia nunca fica negativa: se um efeito a levaria abaixo de zero, ela fica em zero.
7. Há **vitória** quando o jogador entra na saída com pelo menos o mínimo de cristais definido no arquivo de mapa. Se entrar na saída sem cristais suficientes, a partida continua e o jogo informa quantos cristais são exigidos.
8. Há **derrota** quando a energia chega a zero sem vitória, inclusive por efeito de um evento.
9. O jogador pode **desistir** com uma tecla definida pela dupla (por exemplo, `Q`). A desistência leva à tela de fim de jogo.
10. Letras de comando são aceitas em maiúscula e em minúscula.

Ordem das verificações após um movimento válido: (1) mover e descontar a energia; (2) aplicar o efeito da casa; (3) verificar vitória; (4) verificar derrota. Assim, chegar à saída com cristais suficientes vence mesmo que a energia chegue a zero nesse movimento.

Símbolo	Significado
P	posição inicial do jogador
.	caminho livre
#	obstáculo
C	cristal
R	recuperação (+energia)
A	armadilha (–energia)
E	evento definido pela dupla
S	saída

Eventos. A dupla define o valor de cada efeito (por exemplo, R: +5, A: –3) e o efeito do E (por exemplo, revelar a saída ou dar um cristal), e documenta tudo no README.

Vitória alcançável. Recomenda-se exigir entre 50% e 70% dos cristais do mapa (em um mapa com 5 cristais, exigir 3). A energia inicial deve ser generosa: pelo menos cerca de 1,5 vez o número de movimentos do caminho de vitória mais curto.

Mesmo mapa para as duas interfaces. Os símbolos da tabela são os do arquivo de mapa, nas duas opções. Na Raylib, o desenho não precisa mostrar as letras: cada símbolo é convertido em uma forma reconhecível na janela (Anexo B).

A versão-base usa um único mapa, lido de arquivo (Seção 5), e eventos determinísticos. Aleatoriedade, múltiplas fases e inventário são extensões opcionais (Seção 9).

2. Requisitos obrigatórios da versão-base

A versão-base deve conter todos os itens da tabela. Código extra não substitui requisito ausente.

ID	Requisito	Mínimo
R1	Mapa representado em matriz de caracteres.	6 linhas e 8 colunas
R2	Obstáculos que bloqueiam movimento.	6 obstáculos
R3	Cristais coletáveis.	3 cristais
R4	Eventos com efeito observável, ativados uma única vez.	3 eventos, sendo ao menos um `R` e um `A`
R5	Movimentação nas quatro direções.	setas ou teclas documentadas
R6	Controle de limites e obstáculos.	mensagem para movimento inválido
R7	Estado visível durante a partida.	energia, cristais (coletados/mínimo), turnos e última mensagem
R8	Condição de vitória.	saída + mínimo de cristais
R9	Condição de derrota.	energia esgotada
R10	Interface completa em ncurses ou em Raylib.	uma das duas, escolhida pela dupla
R11	Programação estruturada.	8 funções além de `main`, distribuídas entre os módulos
R12	Compilação limpa.	**nenhum aviso** com o comando abaixo
R13	Mapa carregado de arquivo texto, com validação.	`fopen`, `fgets`/`fscanf`; erros tratados (Seção 5)
R14	Tela de fim de jogo com menu.	vitória, derrota e desistência; opções jogar novamente e sair
R15	Organização em módulos.	ao menos 3 arquivos `.c` e um `.h` com include guard
R16	Espaço de tela suficiente.	ncurses: terminal pequeno gera mensagem clara e encerramento; Raylib: janela dimensionada para o mapa

A compilação deve ser feita com o comando da interface escolhida:

```
# ncurses (Linux, WSL e macOS)
gcc -std=c11 -Wall -Wextra -o jogo *.c -lncurses

# Raylib no Linux e no WSL
gcc -std=c11 -Wall -Wextra -o jogo *.c -lraylib -lGL -lm -lpthread -ldl -lrt -lX11

# Raylib no macOS (Homebrew)
gcc -std=c11 -Wall -Wextra -o jogo *.c $(pkg-config --cflags --libs raylib)
```

As bibliotecas no final do comando são obrigatórias: elas ligam o programa à biblioteca escolhida. Sem elas, aparecem erros de ligação como `undefined reference to initscr` (ncurses) ou `undefined reference to InitWindow` (Raylib). No Linux, `-lraylib` sozinho não basta: a Raylib também depende de `GL`, `m`, `pthread`, `dl`, `rt` e `X11`. O README deve registrar a interface escolhida e o comando de compilação usado.

Uso no Code::Blocks (Linux). Crie um projeto *Console Application* em C, adicione os arquivos `.c` e `.h` e confirme que o compilador é o GCC. Em *Project* > *Build options* > *Linker settings*, adicione as bibliotecas do comando acima: `ncurses`, ou `raylib`, `GL`, `m`, `pthread`, `dl`, `rt` e `X11`. Em *Compiler settings* > *Other options*, use `-std=c11 -Wall -Wextra`. A verificação oficial é feita pelo terminal: antes da entrega, compile também com o comando acima.

3. Ambiente de desenvolvimento e correção

A correção será feita nas máquinas dos laboratórios 102 e 104 do prédio 43425, onde também ocorrem as apresentações. Para ncurses, o terminal terá pelo menos 80 colunas por 24 linhas. As aulas práticas de 27/11 e 04/12 servem para testar o jogo nessas máquinas antes da entrega.

Raylib nos laboratórios. Se a Raylib não estiver instalada nos laboratórios, isso não gera desconto: os professores providenciarão uma máquina equivalente para avaliar os trabalhos em Raylib.

Desconto por correção de compilação. O trabalho deve compilar e executar no ambiente de correção com o comando da Seção 2. Se forem necessárias correções no código ou na configuração de compilação sob responsabilidade da dupla para obter o executável, serão descontados **0,25 ponto por erro distinto que tiver de ser corrigido**, conforme registro do diagnóstico e da correção pela banca. Repetições da mesma causa em vários arquivos contam como um erro; avisos que não impedem a compilação não contam como erro de compilação, embora R12 exija compilação sem avisos. O desconto acumulado não reduz a nota abaixo de zero. Problemas da máquina ou falta de uma biblioteca que não foram causados pela dupla não geram desconto. Se o projeto permanecer sem compilar, os critérios que exigem execução não poderão ser comprovados, mas os demais itens continuam sendo avaliados normalmente.

Instalação.

- **ncurses no Linux (Ubuntu/Debian) e no WSL:**
  `sudo apt install build-essential libncurses-dev`
- **Raylib no Ubuntu 24.04 e no WSL:** o Ubuntu 24.04 **não** tem pacote da Raylib. Compile a partir do código-fonte oficial (versão 5.5):

  ```
  sudo apt install build-essential git libasound2-dev libx11-dev libxrandr-dev \
       libxi-dev libxcursor-dev libxinerama-dev libgl1-mesa-dev
  git clone --depth 1 --branch 5.5 https://github.com/raysan5/raylib.git
  cd raylib/src
  make PLATFORM=PLATFORM_DESKTOP
  sudo make install
  ```

  Esse caminho não instala o arquivo do `pkg-config`; por isso, use o comando de compilação com a lista de bibliotecas (Seção 2). Em distribuições mais novas (Debian 13, Ubuntu 25.04 ou posteriores), basta `sudo apt install libraylib-dev`.
- **Windows:** use o WSL (Ubuntu) com as instruções acima. Para Raylib, o WSL só abre janelas no **Windows 11**; no Windows 10, use ncurses ou os laboratórios. O Code::Blocks nativo do Windows não inclui ncurses nem Raylib.
- **macOS:** `brew install ncurses raylib pkg-config`.

Na dúvida sobre a Raylib, prefira ncurses ou consulte os professores antes do checkpoint de 13/11.

4. Interfaces: ncurses ou Raylib

As Figuras 1 a 4 são **apenas ilustrativas**, tiradas de demonstrações com o mapa de exemplo (Seção 5) depois de quatro movimentos para baixo. A dupla não precisa copiar cores, símbolos ou disposição da tela, mas deve cumprir os requisitos da Seção 2.

Nas duas interfaces, o núcleo de regras é o mesmo: leitura e validação do mapa, movimento, obstáculos, cristais, eventos, vitória, derrota, desistência e o menu de repetição. O que muda é apenas a camada de entrada e desenho (Anexos A e B).

Padrão mínimo durante a partida.

- A tela mostra, ao mesmo tempo, o mapa inteiro (incluindo a saída), a posição do jogador, energia, cristais coletados e mínimo exigido, turnos e uma mensagem de estado legível.
- Parede, caminho, cristal, evento e saída são diferenciados por caracteres (ncurses) ou por formas, cores e uma legenda textual (Raylib); não dependa apenas da cor.
- O jogador é desenhado por cima da matriz, sem modificar o símbolo guardado nela.
- Depois de cada comando, válido ou inválido, a tela mostra o estado atual do jogo.

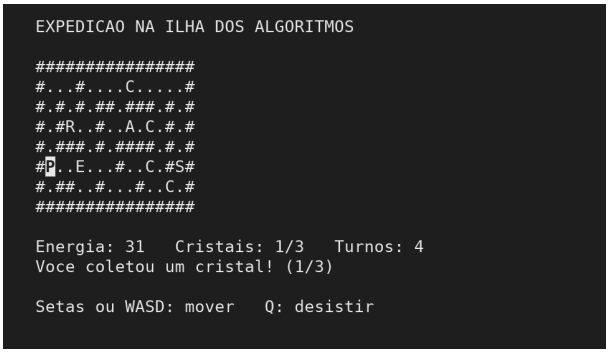


Figura 1: Exploração com ncurses.



Figura 2: Exploração com Raylib.

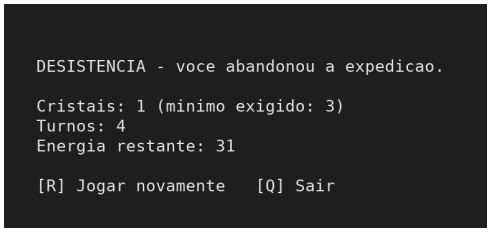


Figura 3: Tela de fim (desistência) com ncurses.

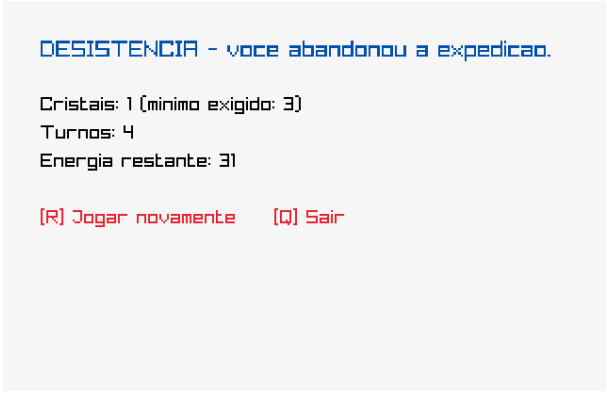


Figura 4: Tela de fim (desistência) com Raylib.

- **Espaço de tela (R16).** ncurses: ao carregar o mapa, verifique se o terminal comporta o mapa e o painel; se não couber, encerre com uma mensagem clara informando o tamanho necessário. Raylib: calcule o tamanho da janela a partir do mapa. Registre no README o tamanho mínimo necessário.

Padrão mínimo da tela final. Ao vencer, perder ou desistir, a tela de exploração é substituída por um resumo com resultado, energia, cristais coletados/mínimo e turnos, e pelo menu `[R] Jogar novamente` e `[Q] Sair`; outras teclas são ignoradas. Na Raylib, fechar a janela pelo X pode encerrar o programa, mas não substitui as opções R/Q na tela final.

Exemplo de fluxo verificável. Iniciar; testar uma parede (nada muda, exceto a mensagem); coletar um cristal (o contador aumenta e a casa esvazia); entrar em `R` e em `A` (efeito uma única vez); tentar a saída cedo demais (a partida continua); voltar com o mínimo exigido (vitória); pressionar outra tecla na tela final (nada acontece); pressionar R (estado inicial restaurado) e Q (encerramento). Teste também desistência e derrota em partidas separadas.

5. Arquivo de mapa

O mapa não pode estar fixo no código-fonte. O jogo deve lê-lo de um arquivo texto no início de cada partida, usando `fopen` e funções como `fgets` e `fscanf`.

Formato.

- **Linha 1:** número de linhas e de colunas do mapa (máximo `MAX_LINHAS` × `MAX_COLUNAS`, definidos em `jogo.h`).
- **Linha 2:** energia inicial e número mínimo de cristais para vencer.
- **Linhas seguintes:** o mapa, com os símbolos da Seção 1, todas com o comprimento informado.

Exemplo (`mapa1.txt`), que pode ser usado nos primeiros testes:

```
8 16
35 3
################
#P..#....C.....#
#.#.#.##.###.#.#
#.#R..#..A.C.#.#
#.###.#.####.#.#
#C..E...#..C.#S#
#.##..#...#..C.#
################
```

Validação obrigatória. Ao carregar, o programa deve:

- verificar se o arquivo foi aberto;
- recusar dimensões acima do máximo ou abaixo do mínimo de R1;
- recusar linhas com comprimento diferente do informado;
- recusar símbolos desconhecidos;
- exigir exatamente um `P` e exatamente um `S`;
- recusar um mínimo de cristais maior que o total de cristais do mapa.

Em qualquer erro, o programa mostra uma mensagem clara e encerra de forma controlada, sem travar nem ter comportamento indefinido. Ao carregar, a posição do `P` é guardada, e a casa passa a ser `.`.

O nome do arquivo pode ser fixo no código ou informado como argumento de linha de comando; as duas formas são aceitas, desde que documentadas no README. A dupla deve entregar pelo menos um mapa próprio, diferente do exemplo acima.

Armadilhas comuns na leitura do mapa

- Depois de `fscanf(arq, "%d %d", ...)`, o `\n` fica no buffer, e o próximo `fgets` lê uma linha vazia. Leia os números com `fgets` + `sscanf`, ou use `fscanf(arq, "%d %d ", ...)` (com espaço no final).
- O buffer do `fgets` precisa de pelo menos `colunas + 2` posições (para o `\n` e o `\0`).
- Arquivos editados no Windows terminam as linhas em `\r\n`. Remova os dois com `linha[strcspn(linha, "\r\n")] = '\0';`
- As linhas da matriz não têm `'\0'` no final: desenhe caractere a caractere, e não com `%s`.
- Sempre chame `fclose` ao terminar a leitura.

6. Tela de fim de jogo

Ao terminar a partida, por vitória, derrota ou desistência, o jogo apresenta uma tela dedicada, diferente da tela de exploração. O programa não fecha logo após o fim da partida: a tela oferece um menu para jogar novamente ou sair. Somente a opção de sair encerra o programa. A tela deve mostrar, no mínimo:

- o resultado: vitória, derrota ou desistência;
- cristais coletados e mínimo exigido;
- turnos jogados e energia restante;
- o menu `[R] Jogar novamente` e `[Q] Sair`. Outras teclas são ignoradas.

Ao jogar novamente, a partida recomeça do zero: o mapa é **recarregado do arquivo**, para que cristais e eventos voltem ao lugar, e energia, cristais, turnos, posição e mensagem voltam aos valores iniciais. Uma implementação possível da função `mostrarFimDeJogo` está no anexo de cada interface.

7. Laço principal

A partida repete: desenhar, ler um comando, validar, mover, processar a casa, atualizar o estado e verificar o fim. Para permitir jogar novamente, o laço da partida fica dentro de um laço externo. O código abaixo é um roteiro para ncurses; a dupla pode usar outras funções, mas deve manter essas etapas separadas. Na Raylib, o laço é o mesmo, com três trocas descritas no Anexo B: `initscr()` vira `InitWindow()`, `endwin()` vira `CloseWindow()` e o par `desenharJogo` + `getch` vira `esperarTecla`.

O mapa é carregado com o ncurses já ativo, inclusive ao jogar novamente. Por isso, em caso de erro, chame

`endwin()` antes do `printf`, para a mensagem aparecer no terminal normal.

```c
#include <ncurses.h>
#include <stdio.h>
#include <string.h>
#include "jogo.h"

#define ARQUIVO_MAPA "mapa1.txt"

int main(void) {
    Jogo jogo;
    int comando, resultado, jogarDeNovo;

    initscr();
    cbreak();
    noecho();
    keypad(stdscr, TRUE);

    do {
        if (!carregarMapa(ARQUIVO_MAPA, &jogo)) {
            endwin();   /* restaura o terminal ANTES do printf */
            printf("Erro ao carregar %s: %s.\n", ARQUIVO_MAPA, jogo.mensagem);
            return 1;
        }
        if (!telaCabe(&jogo)) {
            endwin();
            printf("Terminal pequeno para este mapa: aumente a janela.\n");
            return 1;
        }
        jogo.cristais = 0;
        jogo.turnos = 0;
        jogo.mensagem[0] = '\0';
        resultado = EM_ANDAMENTO;

        while (resultado == EM_ANDAMENTO) {
            desenharJogo(&jogo);
            comando = getch();
            jogo.mensagem[0] = '\0';

            if (comando == 'q' || comando == 'Q') {
                resultado = DESISTENCIA;
            } else if (movimentoValido(&jogo, comando)) {
                moverJogador(&jogo.linhaJ, &jogo.colunaJ, comando);
                jogo.energia--;
                jogo.turnos++;
                processarCasa(&jogo);

                if (chegouNaSaida(&jogo)) {
                    if (jogo.cristais >= jogo.minimoCristais) {
                        resultado = VITORIA;
                    } else {
                        strcpy(jogo.mensagem, "Cristais insuficientes para sair!");
                    }
                }
                if (resultado == EM_ANDAMENTO && jogo.energia <= 0) {
                    resultado = DERROTA;
                }
            } else {
                strcpy(jogo.mensagem, "Movimento invalido!");
            }
        }
        jogarDeNovo = mostrarFimDeJogo(&jogo, resultado);
    } while (jogarDeNovo);

    endwin();
```

```
    return 0;
}
```

Neste roteiro, `carregarMapa` escreve o motivo do erro em `jogo.mensagem`, para que o `printf` mostre uma mensagem clara (por exemplo, *linha 7 do arquivo tem 14 colunas, esperado 16*).

8. Organização em módulos e integração da dupla

A divisão deve ser feita por módulos conectados. Não são aceitos dois programas independentes juntados apenas no final.

Arquivo	Responsabilidade possível
`main.c`	inicialização, laços principais e encerramento
`mapa.c`	carregamento e validação do arquivo, desenho, validação de movimentos
`jogador.c`	leitura de comandos e deslocamento
`eventos.c`	cristais, eventos, saída e tela de fim de jogo
`jogo.h`	constantes, tipo `Jogo` e declarações compartilhadas

Antes de programar, a dupla deve combinar quais funções alteram energia, posição e cristais; quais parâmetros são usados; o que cada função retorna; e qual módulo é chamado em cada etapa do turno. Uma interface possível para `jogo.h`, que agrupa o estado da partida em um registro (`struct`):

```
#ifndef JOGO_H
#define JOGO_H

#define MAX_LINHAS   20
#define MAX_COLUNAS  40
#define TAM_MSG      80

/* resultado da partida */
#define EM_ANDAMENTO 0
#define VITORIA      1
#define DERROTA      2
#define DESISTENCIA  3

typedef struct {
    char mapa[MAX_LINHAS][MAX_COLUNAS];
    int  linhas, colunas;       /* dimensoes lidas do arquivo       */
    int  linhaJ, colunaJ;       /* posicao atual do jogador         */
    int  energia;               /* energia inicial vem do arquivo   */
    int  minimoCristais;        /* lido do arquivo                  */
    int  cristais, turnos;
    char mensagem[TAM_MSG];     /* ultima mensagem mostrada na tela */
} Jogo;

/* mapa.c */
int  carregarMapa(const char *nomeArquivo, Jogo *j); /* 1 = ok, 0 = erro */
int  telaCabe(Jogo *j);          /* ncurses: 1 se mapa + painel cabem */
void desenharJogo(Jogo *j);
int  movimentoValido(Jogo *j, int comando);

/* jogador.c */
void moverJogador(int *linha, int *coluna, int comando);

/* eventos.c */
void processarCasa(Jogo *j);   /* cristal/evento; atualiza mensagem */
int  chegouNaSaida(Jogo *j);
int  mostrarFimDeJogo(Jogo *j, int resultado);
```

```
#endif
```

Se a dupla preferir não usar `struct`, pode passar as variáveis separadamente, por valor ou por ponteiro. Nesse caso, as funções precisam receber também as dimensões do mapa.

Ordem recomendada. Algumas partes dependem de conteúdos vistos mais adiante no semestre: passagem por referência no fim de outubro, registros (`struct`) no início de novembro e arquivos texto a partir de 18/11. Comece pelo que já foi visto:

1. mapa inicializado no código, com o jogador andando e desenhado na interface escolhida;
2. separação em módulos e compilação conjunta; testes de movimentos, obstáculos e limites;
3. cristais, eventos, mensagens, vitória, derrota e tela de fim de jogo;
4. agrupamento do estado em `struct` (a partir de novembro);
5. leitura e validação do mapa a partir do arquivo (a partir de 18/11), que então substitui o mapa fixo;
6. registro dos testes no README.

Os dois integrantes devem revisar o código completo. Cada integrante deve conseguir explicar qualquer parte do programa: como ela recebe dados, o que altera e como devolve o resultado.

9. Extensões opcionais

Extensões só contam como qualidade adicional se estiverem integradas, funcionarem e forem explicadas. Complexidade extra não compensa versão-base incompleta. Sugestões ligadas ao conteúdo da disciplina:

- **Números pseudoaleatórios:** eventos com efeito sorteado (`rand` e `srand`).
- **Recursão:** verificar, ao carregar, se a saída e os cristais são alcançáveis a partir do `P`, com uma função recursiva de preenchimento (*flood fill*).
- **Arquivos binários:** ranking das melhores partidas (menos turnos), salvo em arquivo binário com `fwrite` e lido com `fread`.
- Múltiplos mapas ou fases, modos de dificuldade, inventário.

10. Entregas

10.1 Código-fonte e README.md (dupla). Arquivos `.c` e `.h`, arquivo(s) de mapa e `README.md` contendo: nomes e cartões; interface escolhida e comando de compilação usado; como executar; tamanho mínimo de terminal ou janela; teclas de comando; efeito de cada evento; divisão de responsabilidades; extensões; limitações conhecidas; **um caminho de vitória** para o mapa entregue (sequência de movimentos ou descrição, com a energia restante ao final); e os testes da Seção 12 com os resultados obtidos.

10.2 Relatório técnico em PDF (dupla, 3 a 5 páginas). Foco nas **decisões**, e não na repetição do README: arquitetura e divisão em módulos; escolha das estruturas de dados e justificativa; comunicação entre as funções (o que cada uma recebe, altera e retorna); leitura e validação do arquivo; dificuldades e como foram resolvidas; declaração de uso de IA (Seção 14); referências. Não deve ser uma colagem de código.

10.3 Apresentação em PDF (dupla). De 5 a 8 slides: objetivo, arquitetura, decisões técnicas, demonstração, dificuldades e participação de cada integrante. Os dois apresentam e respondem perguntas sobre o jogo completo.

10.4 Relatório individual (Moodle, até 2 páginas). Tarefas realizadas, funções e módulos implementados, testes executados, dificuldades, contribuição para a integração e reflexão sobre a aprendizagem. Não deve ser cópia do relatório da dupla e não deve atribuir nota ao colega.

Formato. Entregue um único arquivo compactado, com o nome `TF_<cartao1>_<cartao2>.zip`, contendo `src/` (fontes), `mapas/`, `README.md`, `relatorio.pdf` e `apresentacao.pdf`. O relatório individual é entregue em uma tarefa separada do Moodle.

11. Prazos, duplas e atrasos

As datas abaixo valem para **todas as turmas** (A/B e C/D).

Etapa	Data
Apresentação do enunciado	02/10, na aula prática
Formação das duplas no Moodle	até 09/10
Checkpoint em aula (sem nota)	13/11: jogo andando, módulos integrados; `struct` e parâmetros conforme o conteúdo já visto
Conferência da leitura do mapa e da compilação no laboratório	27/11, na aula prática
Conferência final (opcional)	04/12, na aula prática
Entrega do .zip (código, README, relatório técnico e apresentação) e do relatório individual	09/12, 23h59
Apresentações, no horário da aula prática de cada turma	11/12 — por grupo: 7 min de apresentação e demonstração + 5 min de arguição individual (trio: 7,5 min)

- **Atraso:** o código é compilado e lido antes das apresentações. Por isso, entregas atrasadas são aceitas somente até 10/12, 12h, com desconto de 1,0 ponto; depois disso, não são aceitas. Não há atraso para a apresentação, pois a data é fixa.
- **Trios:** somente com autorização do professor, quando a turma tiver número ímpar de alunos. O trio deve dividir o jogo em pelo menos três módulos, um por integrante, e implementar pelo menos uma extensão da Seção 9.
- **Trabalho individual:** somente com autorização do professor, em caso de número ímpar de alunos ou de dupla desfeita. Nesse caso, o critério de integração avalia a organização em módulos e a compilação conjunta.
- **Duplas entre turmas:** permitidas com autorização dos professores; a dupla apresenta em uma das duas sessões.
- **Dupla desfeita:** avise o professor imediatamente. Cada integrante continua individualmente a partir do código existente até aquele momento, e a avaliação considera a situação.
- **Recomendado:** usar um repositório git privado com commits dos dois integrantes. Ele ajuda na integração e serve como evidência de autoria.

12. Testes mínimos

Caso	Ação	Resultado esperado
Carregamento do mapa	iniciar o jogo	mapa do arquivo aparece corretamente na tela
Arquivo ausente	remover ou renomear o arquivo	mensagem clara; programa encerra sem travar; terminal normal
Arquivo malformado	linha curta, símbolo inválido ou dois `P`	mensagem clara; programa encerra sem travar
Tela pequena (ncurses)	iniciar em um terminal menor que o necessário	mensagem clara com o tamanho necessário; programa encerra sem travar
Movimento bloqueado	tentar entrar em `#`	posição, energia e turnos não mudam; mensagem aparece
Limite	tentar sair do mapa	posição, energia e turnos não mudam; mensagem aparece
Coleta	entrar em `C`	cristais aumentam; casa vira `.`
Evento positivo	entrar em `R`	energia aumenta conforme a regra documentada; mensagem aparece
Evento negativo	entrar em `A`	energia diminui (sem ficar negativa); mensagem aparece
Evento consumido	voltar à casa de um evento	nenhum efeito novo
Saída sem requisito	entrar em `S` cedo demais	partida continua; mínimo de cristais é informado
Vitória	chegar a `S` com cristais suficientes	tela de fim mostra vitória
Derrota	gastar a energia até zero	tela de fim mostra derrota
Desistência	pressionar a tecla de desistir	tela de fim mostra desistência
Jogar novamente	escolher `R` (ou `r`)	nova partida com mapa, energia, cristais, turnos e posição reiniciados
Sair	escolher `Q` (ou `q`)	programa encerra; terminal ou janela fecha normalmente

13. Avaliação — 10,0 pontos

Critério	Pontos	Detalhamento
Funcionamento do jogo	2,5	mapa e movimentação (0,7); obstáculos e limites (0,4); energia e turnos (0,3); cristais e eventos (0,4); vitória (0,4); derrota (0,3)
Arquivo de mapa	0,5	leitura correta (0,2); validação e tratamento de erros (0,3)
Interface (ncurses ou Raylib) e tela de fim de jogo	1,0	inicialização, desenho, mensagens e espaço de tela (0,5); tela de fim com jogar novamente, desistência e encerramento (0,5)
Programação estruturada	1,5	funções (0,6); dados, `struct` e parâmetros (0,4); módulos (0,3); laço principal (0,2)
Integração da dupla	1,0	jogo integrado (0,4); compilação conjunta sem avisos (0,3); divisão e comunicação entre módulos (0,3)
README e testes	0,5	compilação, execução, caminho de vitória e casos documentados
Relatório técnico	0,5	decisões, arquitetura, estruturas de dados e referências
Apresentação e demonstração	0,5	clareza, roteiro e participação dos dois
Relatório individual	0,5	contribuição, testes, dificuldades e reflexão
Arguição individual	**1,5**	explicar trechos do código (inclusive do colega) e fazer uma pequena alteração na hora
Total	**10,0**	

Regra de cálculo. Primeiro, atribua os pontos de cada critério pelo desempenho observado. Depois, aplique o desconto de compilação da Seção 3: 0,25 por erro distinto que precisou de correção, com registro do erro e da intervenção. A parcela de 0,3 ponto por "compilação conjunta sem avisos" avalia a qualidade final do projeto e não desconta de novo a mesma falha que já recebeu o desconto de 0,25. A nota final fica entre

0,0 e 10,0. Corrigir um erro para executar os testes não apaga a evidência daquele erro.

O relatório individual e a arguição individual são avaliados por integrante; os demais critérios formam a nota da dupla. Por isso, os integrantes de uma mesma dupla podem ter notas diferentes.

A avaliação prioriza comportamento observável, lógica, organização e capacidade de explicar as decisões. Uma falha isolada em um caso de limite é registrada objetivamente e analisada junto com a solução completa. A vitória deve ser alcançável por um jogador atento em poucas tentativas; um mapa ou requisito de cristais excessivamente difícil deve ser ajustado antes da entrega.

14. Uso de inteligência artificial e integridade acadêmica

O plano de ensino proíbe o uso de IA em atividades avaliativas, salvo autorização explícita e guiada do professor. **Esta seção é essa autorização, e ela vale somente nos limites abaixo.** O objetivo do trabalho é que a dupla aprenda a programar, e não que entregue um programa pronto.

Permitido (como apoio ao estudo):

- tirar dúvidas pontuais sobre a linguagem C ou sobre funções da ncurses ou da Raylib;
- entender uma mensagem de erro do compilador;
- pedir a explicação de um conceito.

Não permitido:

- fornecer este enunciado, ou partes dele, a ferramentas de IA;
- usar IA para gerar o código do jogo, módulos, funções, o README, os relatórios ou a apresentação.

Todo código entregue deve ter sido escrito pela dupla, e cada integrante deve saber explicar qualquer trecho, inclusive a parte feita pelo colega. Quem usar IA como apoio deve informar, no relatório técnico, qual ferramenta usou, para quê e em quais partes.

Verificação.

- As entregas passam por ferramenta de detecção de similaridade e são comparadas com os trabalhos anteriores de cada aluno.
- Na apresentação, os professores podem pedir que cada integrante explique trechos do código ou faça pequenas alterações na hora.
- Havendo suspeita, a dupla é convocada para uma arguição adicional.
- Recebe nota zero no trabalho o integrante que não conseguir explicar ou modificar o código entregue, ou quando houver evidência de código gerado por IA ou copiado de colegas, de trabalhos anteriores ou da internet. A penalidade vale para todos os envolvidos.

Além da penalidade na disciplina, fraude e plágio, inclusive com o uso de IA, constituem infração disciplinar segundo o Código Disciplinar Discente da UFRGS. As sanções vão de advertência a desligamento da Universidade.

Anexo A. Interface com ncurses

Este anexo vale apenas para a dupla que escolher ncurses.

O terminal como grade. Reserve uma linha e uma coluna do terminal por casa do mapa. Desenhe a matriz com `mvaddch`, sobreponha o `P` apenas ao desenhar e deixe linhas abaixo do mapa para energia, cristais, turnos, mensagem e teclas. A cada atualização, chame `clear()`, desenhe tudo e chame `refresh()` uma vez. Use `getch()` para ler as teclas. O programa deve inicializar e encerrar o terminal corretamente:

```c
#include <ncurses.h>

int main(void) {
    initscr();              /* inicia a tela do ncurses          */
    cbreak();               /* teclas sem esperar Enter           */
    noecho();               /* nao mostra a tecla digitada        */
    keypad(stdscr, TRUE);   /* habilita as setas (KEY_UP etc.)    */

    /* partidas */

    endwin();               /* restaura o terminal                */
    return 0;
}
```

Função	Uso
`clear()`	limpa a tela
`mvprintw(linha, coluna, formato, ...)`	escreve em uma posição (como `printf`)
`mvaddch(linha, coluna, caractere)`	escreve um caractere em uma posição
`refresh()`	mostra na tela o que foi desenhado
`getch()`	espera uma tecla; setas chegam como `KEY_UP`, `KEY_DOWN`, `KEY_LEFT`, `KEY_RIGHT`
`getmaxyx(stdscr, lin, col)`	guarda em `lin` e `col` o tamanho atual do terminal

Redesenho e mensagens. A cada turno, `desenharJogo` limpa a tela, desenha o mapa, desenha `P` na posição do jogador sem alterar a matriz, e mostra a linha de estado (energia, cristais coletados/mínimo, turnos) e a última mensagem. Como a tela é limpa a cada turno, uma mensagem impressa fora de `desenharJogo` some antes de ser lida. Por isso, guarde a mensagem no campo `mensagem` do estado do jogo (Seção 8) e deixe `desenharJogo` exibi-la.

Espaço de tela (R16). Depois de carregar o mapa, a função `telaCabe` usa `getmaxyx` para comparar o tamanho do terminal com o espaço necessário (linhas do mapa + linhas do painel; a maior largura entre o mapa e o painel). Se não couber, o `main` encerra com uma mensagem clara (Seção 7). Não é preciso tratar o redimensionamento durante a partida.

Chame `endwin()` antes de qualquer saída definitiva do programa, inclusive por erro. O jogo não deve depender de `printf` para a interface principal.

Tela de fim de jogo. Uma organização possível:

```c
/* retorna 1 para jogar novamente e 0 para sair */
int mostrarFimDeJogo(Jogo *j, int resultado) {
    int tecla;

    clear();
    if (resultado == VITORIA) {
        mvprintw(2, 2, "VITORIA! Voce escapou da ilha.");
    } else if (resultado == DERROTA) {
        mvprintw(2, 2, "DERROTA - sua energia acabou.");
    } else {
        mvprintw(2, 2, "DESISTENCIA - voce abandonou a expedicao.");
    }
    mvprintw(4, 2, "Cristais: %d (minimo exigido: %d)",
```

```
            j->cristais, j->minimoCristais);
    mvprintw(5, 2, "Turnos: %d", j->turnos);
    mvprintw(6, 2, "Energia restante: %d", j->energia);
    mvprintw(8, 2, "[R] Jogar novamente   [Q] Sair");
    refresh();

    do {
        tecla = getch();
    } while (tecla != 'r' && tecla != 'R' &&
             tecla != 'q' && tecla != 'Q');

    return tecla == 'r' || tecla == 'R';
}
```

Anexo B. Interface com Raylib

Este anexo vale apenas para a dupla que escolher Raylib. Antes de começar, instale a biblioteca e compile um programa de teste (Seção 3).

A diferença principal: quadros, e não espera. No ncurses, `getch()` para o programa até chegar uma tecla, e a tela é desenhada uma vez por turno. Na Raylib não existe essa espera: a janela precisa ser redesenhada **a cada quadro** (cerca de 30 vezes por segundo), entre `BeginDrawing()` e `EndDrawing()`, e as teclas são consultadas em cada quadro. Se o desenho for feito só uma vez por turno, a janela pisca ou fica em branco.

Para que o resto do jogo continue igual ao do ncurses, use uma função que **faz o papel do `getch()`**: ela redesenha a tela a cada quadro até uma tecla ser pressionada e então a devolve. Uma implementação possível, em `jogador.c`:

```c
/*
 * Redesenha a tela a cada quadro ate uma tecla ser pressionada e a retorna.
 * telaFinal = 0 desenha a exploracao; 1 desenha a tela de fim de jogo.
 * Se a janela for fechada pelo X, retorna KEY_Q (sair).
 */
int esperarTecla(Jogo *j, int telaFinal, int resultado) {
    int tecla;

    while (!WindowShouldClose()) {
        BeginDrawing();
        if (telaFinal) {
            desenharFimDeJogo(j, resultado);
        } else {
            desenharJogo(j);
        }
        EndDrawing();

        tecla = GetKeyPressed();   /* letras chegam sempre em maiuscula */
        if (tecla != 0) {
            return tecla;
        }
    }
    return KEY_Q;
}
```

Com ela, o laço principal da Seção 7 muda em três pontos:

- `initscr()` e as chamadas seguintes viram a abertura da janela. Carregue o mapa uma vez *antes* de abrir a janela, porque o tamanho dela depende do mapa:

  ```c
  InitWindow(larguraJanela(&jogo), alturaJanela(&jogo),
             "Expedicao na Ilha dos Algoritmos");
  SetExitKey(KEY_NULL);   /* sem isso, a tecla ESC fecha a janela */
  SetTargetFPS(30);       /* quadros por segundo, e nao turnos */
  ```

- As duas linhas `desenharJogo(&jogo);` e `comando = getch();` viram uma só:

  ```c
  comando = esperarTecla(&jogo, 0, 0);   /* desenha e espera uma tecla */
  ```

  As comparações com `'q'` e `'Q'` continuam funcionando, porque `KEY_Q` vale `'Q'`.

- `endwin()` vira `CloseWindow()`, inclusive antes do `printf` de erro. O teste de tela (`telaCabe`) não é necessário: a janela já é criada do tamanho do mapa.

As setas chegam como `KEY_UP`, `KEY_DOWN`, `KEY_LEFT` e `KEY_RIGHT`, com os mesmos nomes do ncurses. Por isso, um `switch` com `case KEY_UP:` funciona nas duas bibliotecas. As letras chegam como `KEY_W`, `KEY_A` etc., sempre em maiúscula.

Tamanho da janela e das casas. Escolha um tamanho fixo de casa em pixels (por exemplo, 28) e calcule a posição de cada casa: `x = margem + coluna * tamanho` e `y = margem + linha * tamanho`. Reserve uma área abaixo do mapa para o painel e a legenda. Com casas de 28 pixels, o maior mapa permitido (20

× 40) gera uma janela de cerca de 1160 × 750 pixels; em telas menores, use casas de 24 pixels.

Conversão dos símbolos. O arquivo de mapa segue exatamente o formato da Seção 5. Ao desenhar cada casa, converta o símbolo em um elemento visual identificável. A tabela mostra uma representação possível (a das figuras da Seção 4); cores e desenhos são livres, mas os significados e efeitos são os mesmos.

Mapa	Significado	Amostra	Sugestão
`P`	jogador		quadrado `MAGENTA` com P, sobre o piso
`.`	caminho livre		quadrado `LIGHTGRAY`
`#`	obstáculo		quadrado `DARKGRAY`
`C`	cristal		círculo `GOLD` sobre o piso
`R`	recuperação		quadrado `DARKGREEN` com +
`A`	armadilha		quadrado `RED` com !
`E`	evento da dupla		quadrado `PURPLE` com ?
`S`	saída		quadrado `BLUE` com S

O `P` é lido uma vez para definir a posição inicial; a casa passa a ser caminho livre na matriz, e o jogador é desenhado por cima da casa onde estiver. Um `C`, `R`, `A` ou `E` consumido vira caminho livre também no desenho. Mostre uma legenda textual que permita reconhecer os elementos sem depender apenas da cor. A colisão é decidida pelo símbolo da matriz, e não pela cor do desenho.

Função	Uso
`InitWindow(largura, altura, titulo)` / `CloseWindow()`	abre e fecha a janela
`BeginDrawing()` / `EndDrawing()`	delimitam o desenho de um quadro
`ClearBackground(cor)`	limpa a janela com uma cor de fundo
`DrawRectangle(x, y, largura, altura, cor)`	desenha um retângulo (uma casa do mapa)
`DrawCircle(x, y, raio, cor)`	desenha um círculo (um cristal)
`DrawText(texto, x, y, tamanho, cor)`	escreve texto (painel, mensagem, legenda)
`TextFormat(formato, ...)`	monta um texto como o `printf`, para usar no `DrawText`
`GetKeyPressed()`	devolve a tecla pressionada neste quadro, ou 0
`WindowShouldClose()`	verdadeiro quando o usuário fecha a janela pelo X

Redesenho e mensagens. `desenharJogo` limpa o fundo, desenha o mapa casa por casa, desenha o jogador e mostra o painel de estado (energia, cristais coletados/mínimo, turnos) e a última mensagem, guardada no mesmo campo `mensagem` usado no ncurses. Ela não chama `BeginDrawing` nem `EndDrawing`: quem faz isso é `esperarTecla`.

Tela de fim de jogo. `desenharFimDeJogo` desenha o resumo e o menu, como na versão ncurses, e `mostrarFimDeJogo` espera R ou Q:

```
/* retorna 1 para jogar novamente e 0 para sair */
int mostrarFimDeJogo(Jogo *j, int resultado) {
    int tecla;

    do {
        tecla = esperarTecla(j, 1, resultado);
    } while (tecla != KEY_R && tecla != KEY_Q);

    return tecla == KEY_R;
}
```